

Reinforcement Learning

Dynamic Programming Sutton/Barto* Chapter 4

Michael Hahsler, SMU

With figures from Sutton/Barto*

*Sutton and Barto, Reinforcement Learning: An Introduction,
2nd edition, MIT Press, Cambridge, MA, 2018



Goal

- Compute the optimal policy π_* given a perfect finite MDP model.
- Bellman optimality equations:

$$v_*(s) = \max_a \mathbb{E}_\pi [R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a]$$

or

$$q_*(s, a) = \max_a \mathbb{E}_\pi \left[R_{t+1} + \gamma \underbrace{\max_{a'} q_*(S_{t+1}, a')}_{= v_*(S_{t+1})} \middle| S_t = s, A_t = a \right]$$

- **Idea:** turn the Bellman optimality equations into update rules.
- **Issues:** The estimate for S_t depends on the current estimates of the successor states S_{t+1} . This is a recursion!

Summary of Notation

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\mathbf{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\mathbf{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bold)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X , i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\operatorname{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
Q, Q_t	array estimates of action-value function q_π or q_*
$\bar{V}_t(s)$	expected approximate action value
U_t	target for estimate at time t

MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r s, a)$	probability of transition to state s' and receiving reward r , from state s taking action a
	expected immediate reward from state s after action a
$\pi(a s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

Tabular Methods

- Dynamic programming is a tabular method.
- The value functions v or q are stored in arrays (tables).
- Tabular methods can often find the exact solution (i.e., optimal value function and policy).
- **Issue:** Since the table for v has $|\mathcal{S}|$ and for q has $|\mathcal{S}| \times |\mathcal{A}|$ many entries, this is only feasible for problems with a small state space!
- We will later talk about approximate methods that can work with large state spaces.

Policy Evaluation

Policy Evaluation (Prediction)

- First step: Compute the state-value function v_π for a given policy π .
- In dynamic programming (DP) this is called policy evaluation.

- Recursive relationship:

$$\begin{aligned} v_\pi(s) &\stackrel{\text{def}}{=} \mathbb{E}_\pi[G_t | S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')] \end{aligned}$$

- $|S|$ equations, one for each state s .
- To calculate $v(s)$ from later $v(s')$ is called **bootstrapping** in DP.
- **Guarantee:** A unique v_π if $\gamma < 1$ or termination is guaranteed by π

Solution Options

1. Solve the system of $|\mathcal{S}|$ linear equations.
This is hard if $|\mathcal{S}|$ is very large.

2. Iterative Policy Evaluation

- Start with arbitrary values for all $v(s)$
- Update all values at iteration k using

$$\begin{aligned} v_{k+1}(s) &\stackrel{\text{def}}{=} [R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s] \\ &= \sum_a \pi(a|s) \underbrace{\sum_{s',r} p(s',r | s, a) [r + \gamma v_k(s')]} \end{aligned}$$

In DP called “expected update.”
Uses the expectation over all possible next states.

- $v_k = v_\pi$ is a fixed point. Updates converge for $k \rightarrow \infty$

Alg. Iterative Policy Evaluation

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\text{terminal})$ to 0

Loop:

$\Delta \leftarrow 0$

 Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Policy Iteration

Policy Improvement

- **Goal:** Find a better policy.
- **Policy improvement theorem:** If

$$q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s) \quad \forall s \in \mathcal{S}$$

then π' is at least as good or better than π , i.e.,

$$v_{\pi'}(s) \geq v_{\pi}(s) \quad \forall s \in \mathcal{S}.$$

- **Policy improvement:** create a new policy π' by choosing in each state the best (aka greedy) action using $q_{\pi}(s, a)$:

$$\pi'(s) = \underset{a}{\operatorname{argmax}} q_{\pi}(s, a)$$

- **Guarantee:** This gives a strictly better policy π' unless π is already optimal.

Policy Iteration

- Sequence of monotonically improving policies:

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} v_{\pi_2} \xrightarrow{I} \dots \pi_* \xrightarrow{E} v_*$$

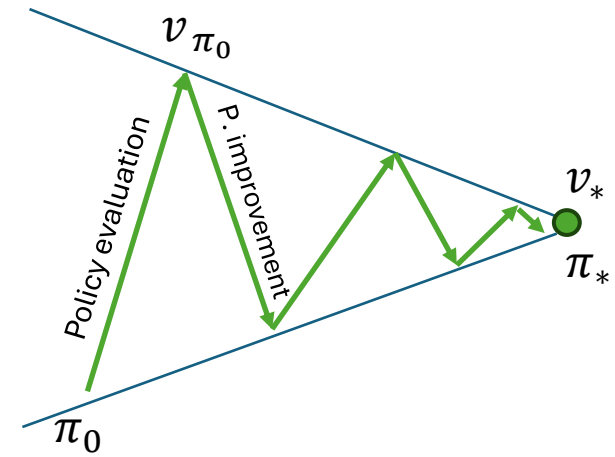
$\xrightarrow{E}$ policy eval. step
 $\xrightarrow{I}$ policy improvement step

Guarantees:

- Policy improvement is guaranteed to improve the policy + finite MDPs have a finite number of policies $\rightarrow$ Policy Iteration must **converge!**

Notes:

- Policy becomes stable before v_* is reached.
- Policy iteration typically converges very fast (a few iterations)
- Issue: Policy estimation is computationally expensive!



Alg. Policy Iteration

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\text{terminal}) \doteq 0$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ *true*

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Value Iteration

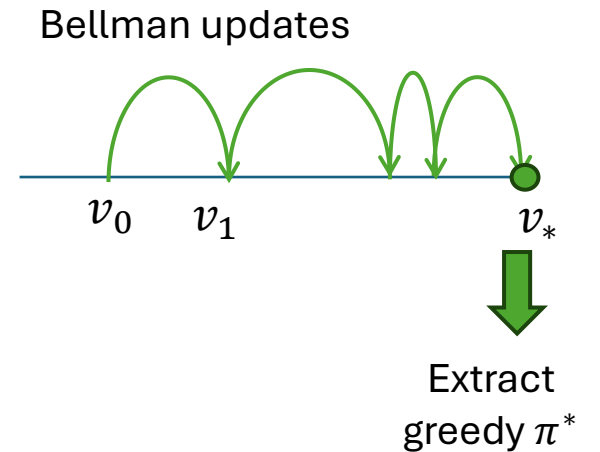
Value Iteration

- Combines the policy improvement and truncated policy evaluation steps:

$$\begin{aligned} v_{k+1}(s) &\stackrel{\text{def}}{=} \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')] \end{aligned}$$

$\max_a$... policy improvement
 $\mathbb{E}$... policy evaluation

- Guarantee:** Sequence $\{v_k\}$ converges to v_*
- Practical termination when $v_{k+1}(s) - v_k(s) < \theta \quad \forall s \in \mathcal{S}$



Alg. Value Iteration

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
```

until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

Issues with DP Methods

Space

- Tabular method: stores the complete value function as table V .

Time

- Updates repeatedly sweep over the whole state space.
- Wastes computation by updating states that are not relevant for the optimal behavior.

GPL: Generalized Policy Iteration

GPI: Generalized Policy Iteration

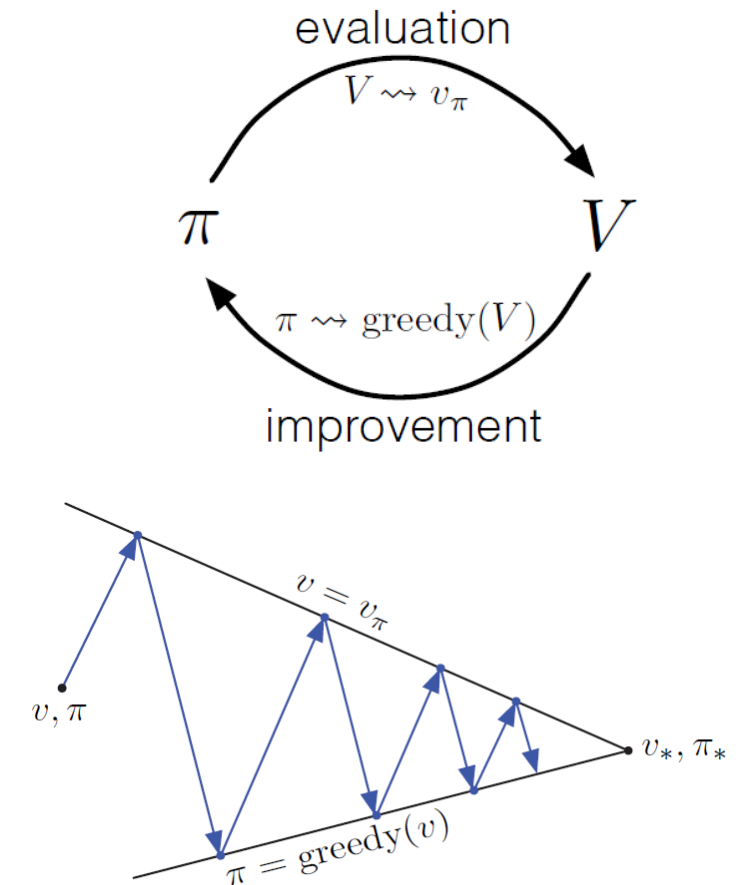
Steps

- **Policy evaluation:** make the value function consistent with the current policy π .
- **Policy improvement:** make policy greedy with respect to the current value function estimate V .

Most DP methods can be described as a special case of GPI:

- **Policy iteration:** each eval and improvement step is completed (by a sweep over $\mathcal{S}$) before moving on.
- **Value iteration:** uses a single iteration for the policy iteration.
- **Asynchronous DP:** Instead of sweeps over $\mathcal{S}$, updates states in some other order. This interleaves pol. eval. and pol. Improvement.

All three approaches typically converge to the optimal solution.



Efficiency

- **Direct Search:** Search space is exponential in the number of states $|\mathcal{S}|$ and actions $|\mathcal{A}|$.
- **LP (linear program):** solve a system of $|\mathcal{S}|$ equations with $|\mathcal{S}|$ state variables and $|\mathcal{S}| \times |\mathcal{A}|$ constraints.
- **DP:** polynomial in $|\mathcal{S}|$ and $|\mathcal{A}|$.

Search and LP are impractical. **DP can be used today with millions of states.**

Summary

- DP needs access to the **model**'s function p .
- **Prediction**: Calculates the value function for a given policy.
- **Control**: Calculates the optimal policy for an MDP.
- DP uses the recursive relationship between state values called the Bellman optimality equations.
- It bootstraps, i.e., it calculates $v(s)$ from later $v(s')$
- The methods store the complete value function as a table and iteratively sweep over the table.
- The methods can be generalized as the GPI approach.